Time Series Analysis

Part 1: Lecture 2

Various Topics

Outline

1 Stationarity and differencing

2 Transformations and adjustments

3 Moving averages

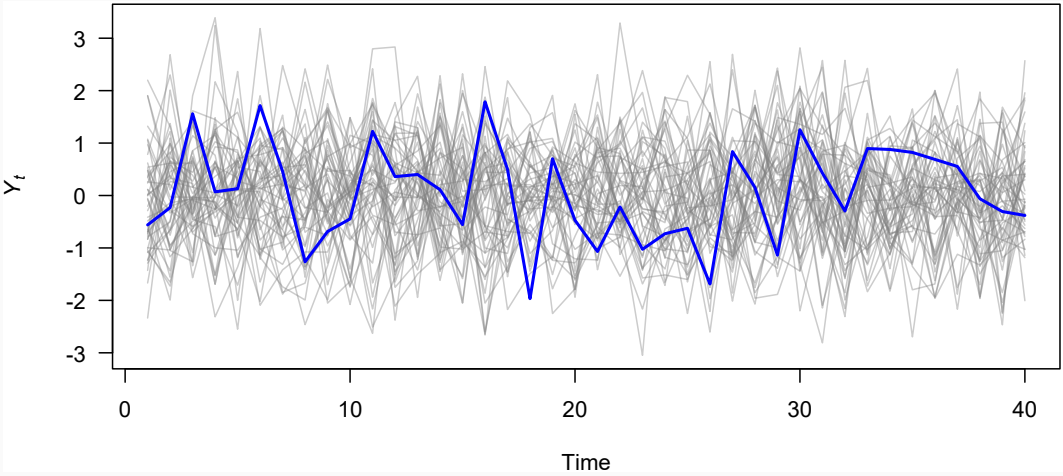
4 Classical Decomposition

5 A tidy forecasting workflow

6 Benchmark methods

Recall: Time series model

A *time series model* for $\{y_t\}$ is a specification of the *joint distributions* of a sequence of *random variables* $\{X_t\}$, of which $\{y_t\}$ is thought to be a realisation



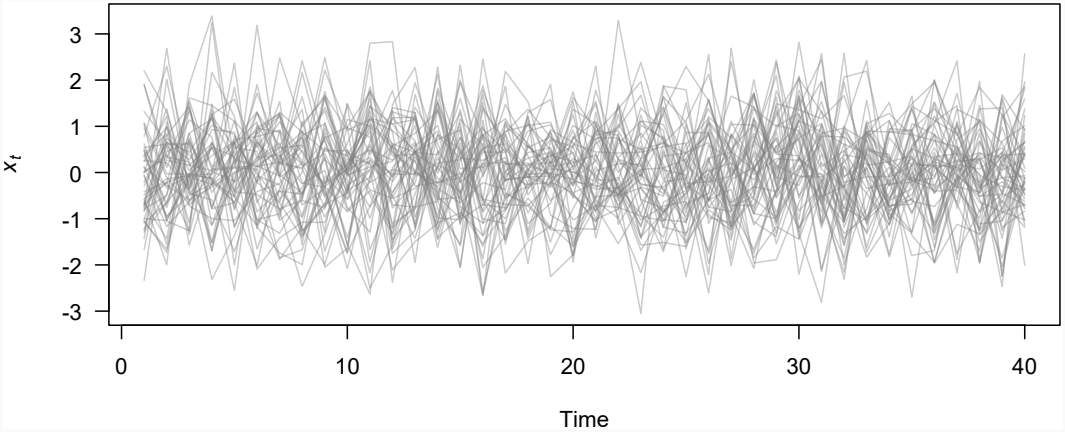
Recall: Time series model

Definition

$$\bar{y}(t) \equiv \frac{1}{n}\sum_{i=1}^{n} y^{(i)}(t) \quad \rightarrow \quad \mathbb{E}\left[Y(t)\right] \tag{1}$$

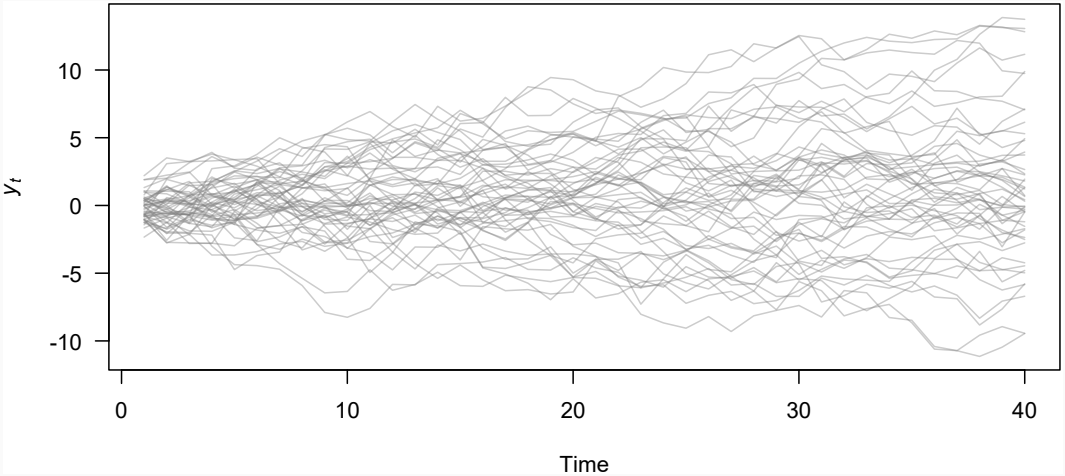
$$\frac{1}{n}\sum_{i=1}^{n} (y^{(i)}(t) - \bar{y}(t))(y^{(i)}(s) - \bar{y}(s)) \quad \rightarrow \quad \mathrm{Cov}\left[Y(t), Y(s)\right] \tag{2}$$

Some simple time series models



(Gaussian) White noise: $y_t \sim N(0, 1)$ - values are (1) independent and (2) identically distributed with a mean of zero

Some simple time series models



Random walk: $y_t = y_{t-1} + \varepsilon_t$, with $\varepsilon_t \sim N(0, 1)$

Stationarity

Definition

If $\{y_t\}$ is a stationary time series, then for all s, the distribution of $(y_t, \ldots, y_{t+s})$ does not depend on t.

Strict stationarity

A time series, $y_1, y_2, ..., y_t, ..., y_T$, is called strictly stationary if the distributions of

$$(y_t, y_{t+1}, ..., y_{t+s}) \text{ and } (y_{t+h}, y_{t+1+h}, ..., y_{t+s+h})$$

are the same for all h. The distribution of y_t does not depend on t.

Weak stationarity

The time series is called weakly stationary if

$E(y_t) = \mu$; $V(y_t) = E((y_t - \mu)^2) = \gamma_0$; $Cov(y_t, y_{t-h}) = E((y_t - \mu)(y_{t-h} - \mu)) = \gamma_h$ for $h = 1, 2, ...$.

Stationarity

- Strict stationarity implies that $y_t (t = 1, 2, ..., T)$ are random draws from the same unconditional distribution.
- Weak stationary is only a statement about the mean and the variance (the first two moments of the distribution) and y_t fluctuates around a constant level: equilibrium correcting / mean reverting.

Stationarity

A stationary series is a time series where there are no changes in the underlying system (a series whose properties do not depend on the time at which the series is observed)

- Constant mean (no trend / roughly horizontal)
- Constant variance (no heteroscedacticity)
- Constant autocorrelation structure
- No periodic variations or seasonality

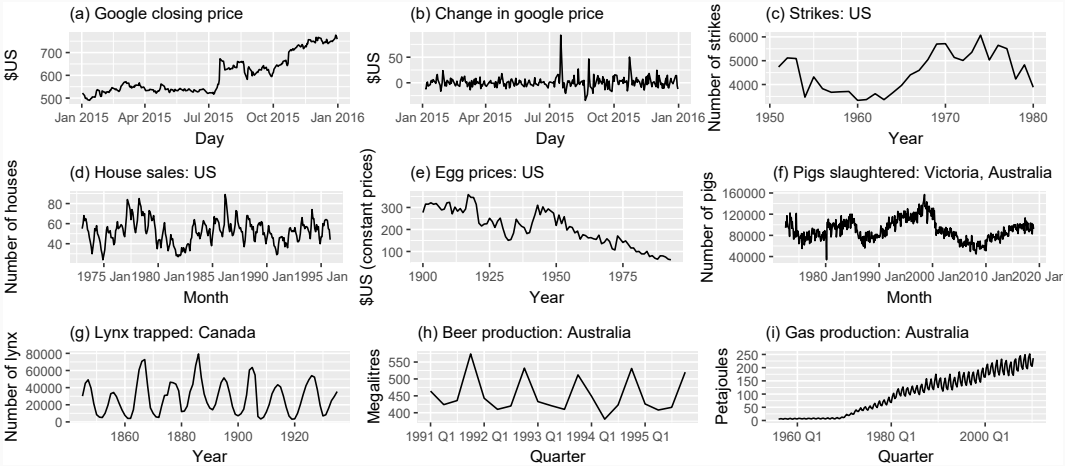
Some cases can be confusing!

In general, No patterns predictable in the long-term

Identifying non-stationary series

- Summary statistics (split your data into chunks where possible)
- Histogram plot
- Time plot
- The ACF plot
  - The ACF of stationary data drops to zero relatively quickly
  - The ACF of non-stationary data decreases slowly
  - For non-stationary data, the value of r_1 is often large and positive
- Statistical tests (Unit root tests: Augmented Dickey-Fuller test, KPSS test - discussed later!!)

Stationary or not



Why Is Stationarity important?

- Some time series models (e.g., autoregressive models) require a stationary time series
- Stationary time series are easier to model due to their constant statistical properties

Dealing with non-stationarity

- Transformations
  - Remove trend (constant mean)
  - Remove heteroscedasticity (use transformations such as logarithms to achieve constant variance)
  - Remove autocorrelation with differencing (exploit constant structure)
  - Remove seasonality (no periodic component).
- Oftentimes you will have to do several of these on one dataset!

Differencing

- Differencing helps to stabilise the mean.
- The differenced series is the change between each observation in the original series: $y'_t = y_t - y_{t-1}$.
- The differenced series will have only $T - 1$ values since it is not possible to calculate a difference y'_1 for the first observation.

Example: Google stock price

```
google_2018 <- gafa_stock |>
  filter(Symbol == "GOOG", year(Date) == 2018)

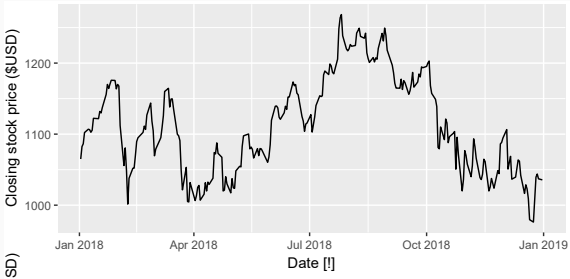
g1 <- google_2018 |>
  autoplot(Close) +
  labs(y = "Closing stock price ($USD)")

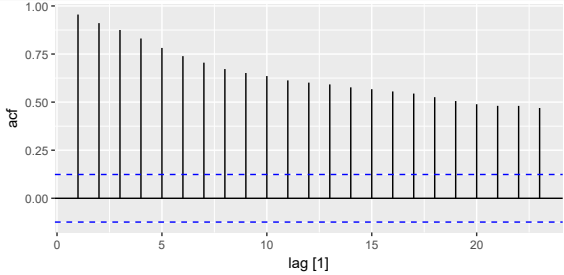
g2 <- google_2018 |>
  ACF(Close) |>
  autoplot()

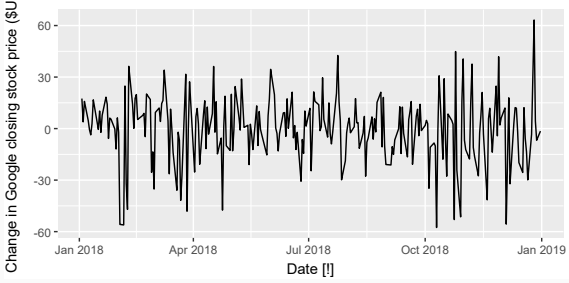
g3 <- google_2018 |>
  autoplot(difference(Close)) +
  labs(y = "Change in Google closing stock price ($USD)")

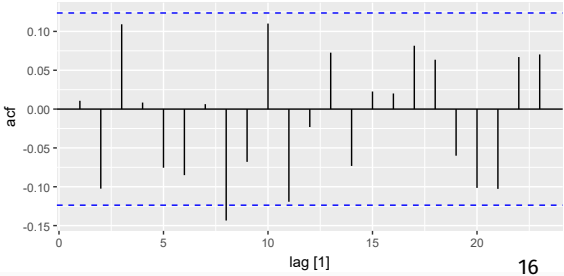
g4 <- google_2018 |>
  ACF(difference(Close)) |>
  autoplot()
```

Example: Google stock price









Second-order differencing

Occasionally the differenced data will not appear stationary and it may be necessary to difference the data a second time:

$$\begin{aligned} y''_t &= y'_t - y'_{t-1} \\ &= (y_t - y_{t-1}) - (y_{t-1} - y_{t-2}) \\ &= y_t - 2y_{t-1} + y_{t-2}. \end{aligned}$$

- y''_t will have $T - 2$ values.
- In practice, it is almost never necessary to go beyond second-order differences.

Seasonal differencing

A seasonal difference is the difference between an observation and the corresponding observation from the previous year.

$$y'_t = y_t - y_{t-m}$$

where m = number of seasons.

- For monthly data $m = 12$.
- For quarterly data $m = 4$.
- Seasonally differenced series will have $T - m$ obs.

Antidiabetic drug sales

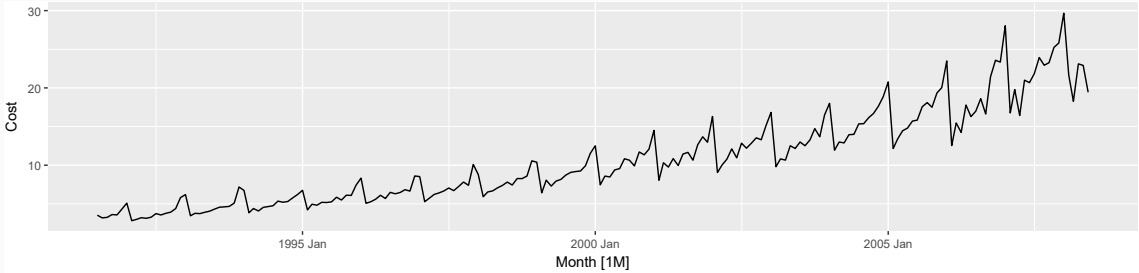
```
a10 <- PBS |>
  filter(ATC2 == "A10") |>
  summarise(Cost = sum(Cost) / 1e6)

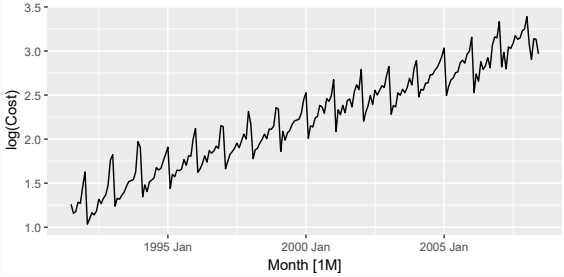
asales1<- a10 |>
  autoplot(Cost)

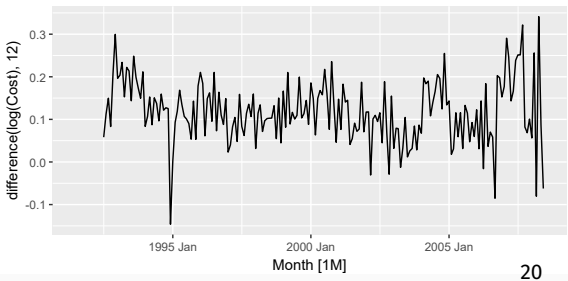
asales2 <- a10 |>
  autoplot( log(Cost))

asales3 <- a10 |> autoplot(
  log(Cost) |> difference(12)
)
```

Antidiabetic drug sales







Corticosteroid drug sales

```
h02 <- PBS |>
  filter(ATC2 == "H02") |>
  summarise(Cost = sum(Cost) / 1e6)

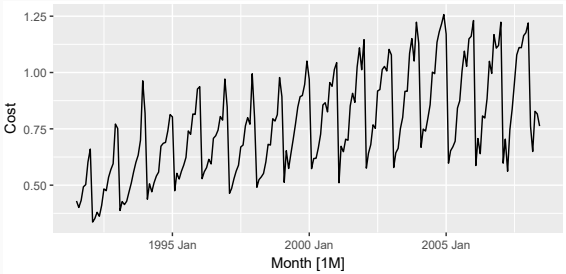
h1 <- h02 |>
  autoplot( Cost)

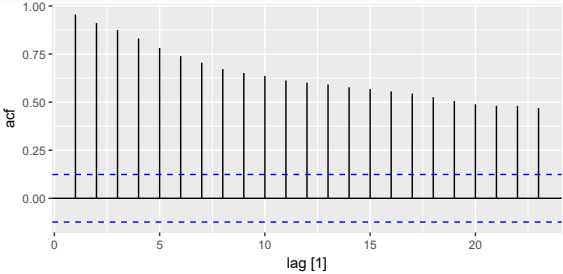
h3 <- h02 |>
  autoplot(log(Cost))

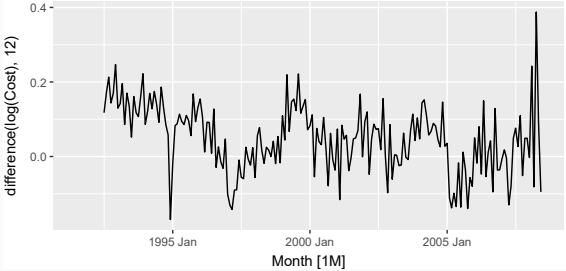
h3 <- h02 |>
  autoplot( log(Cost) |> difference(12))

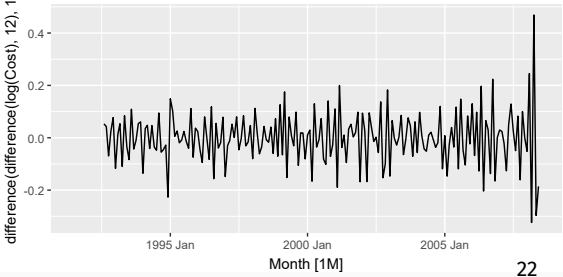
h4 <- h02 |>
  autoplot( log(Cost) |> difference(12) |> difference(1))
```

Corticosteroid drug sales









Corticosteroid drug sales

- Seasonally differenced series is closer to being stationary.
- Remaining non-stationarity can be removed with further first difference.

If $y'_t = y_t - y_{t-12}$ denotes seasonally differenced series, then twice-differenced series is

$$\begin{aligned} y^*_t &= y'_t - y'_{t-1} \\ &= (y_t - y_{t-12}) - (y_{t-1} - y_{t-13}) \\ &= y_t - y_{t-1} - y_{t-12} + y_{t-13}\,. \end{aligned}$$

Seasonal differencing

When both seasonal and first differences are applied…

- it makes no difference which is done first—the result will be the same.
- If seasonality is strong, seasonal differencing be done first because sometimes the resulting series will be stationary and there will be no need for further first difference.

It is important that if differencing is used, the differences are interpretable.

Interpretation of differencing

- first differences are the change between one observation and the next;
- seasonal differences are the change between one year to the next.

But taking lag 3 differences for yearly data, for example, results in a model which cannot be sensibly interpreted.

Unit root tests: Augmented Dickey-Fuller (ADF) test

Unit root tests: Statistical tests to determine the required order of differencing.

For the ADF test, the following regression model is estimated:

$$y'_t = \phi y_{t-1} + \beta_1 y'_{t-1} + \beta_2 y'_{t-2} + \cdots + \beta_k y'_{t-k},$$

where y'_t denotes the first-differenced series, $y'_t = y_t - y_{t-1}$ and k is the number of lags to include in the regression.

If the original series, y_t, needs differencing, then the coefficient $\hat{\phi}$ should be approximately zero. If y_t is already stationary, then $\hat{\phi} < 0$.

The null-hypothesis for an ADF test is that the data are non-stationary. So large p-values are indicative of non-stationarity, and small p-values suggest stationarity. Using the usual 5% threshold, differencing is required if the p-value is greater than 0.05.

Unit root tests: The KPSS test

The Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test:

This reverses the hypotheses, so the null-hypothesis is that the data are sstationary. In this case, small p-values (e.g., less than 0.05) suggest that differencing is required.

Example: Google stock price: KPSS test

```
google_2018 <- gafa_stock |>
  filter(Symbol == "GOOG", year(Date) == 2018) |>
  features(Close, unitroot_ndiffs)
```

Automatically selecting differences

STL decomposition: $y_t = T_t + S_t + R_t$

Seasonal strength $F_s = \max\left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(S_t+R_t)}\right)$

If $F_s > 0.64$, do one seasonal difference.

```
h02 |> mutate(log_sales = log(Cost)) |>
  features(log_sales, feat_stl)
```

```
# A tibble: 1 x 9
  trend_strength seasonal_strength_year seasonal_peak_year
           <dbl>                  <dbl>              <dbl>
1          0.957                  0.955                  6
# i 6 more variables: seasonal_trough_year <dbl>, spikiness <dbl>,
#   linearity <dbl>, curvature <dbl>, stl_e_acf1 <dbl>, stl_e_acf10 <dbl>
```

Automatically selecting differences

```
h02 |> mutate(log_sales = log(Cost)) |>
  features(log_sales, unitroot_nsdiffs)
```

```
# A tibble: 1 x 1
  nsdiffs
    <int>
1       1
```

```
h02 |> mutate(d_log_sales = difference(log(Cost), 12)) |>
  features(d_log_sales, unitroot_ndiffs)
```

```
# A tibble: 1 x 1
  ndiffs
   <int>
1      1
```

Backshift notation

A very useful notational device is the backward shift operator, B, which is used as follows:

$$By_t = y_{t-1}$$

In other words, B, operating on y_t, has the effect of **shifting the data back one period**.

Two applications of B to y_t **shifts the data back two periods**:

$$B(By_t) = B^2y_t = y_{t-2}$$

For monthly data, if we wish to shift attention to "the same month last year", then B^{12} is used, and the notation is

Backshift notation

The backward shift operator is convenient for describing the process of *differencing*.

A first-order difference can be written as

$$y'_t = y_t - y_{t-1} = y_t - By_t = (1 - B)y_t$$

Similarly, if second-order differences (i.e., first differences of first differences) have to be computed, then: [y' '{t} = y{t} - 2y_{t - 1} + y_{t - 2} = (1 - B)^{2} y_{t}]

Backshift notation

- Second-order difference is denoted $(1-B)^2$.
- *Second-order difference* is not the same as a *second difference*, which would be denoted $1-B^2$;
- In general, a dth-order difference can be written as

$$(1-B)^d y_t$$

- A seasonal difference followed by a first difference can be written as

$$(1-B)(1-B^m)y_t$$

Backshift notation

The “backshift” notation is convenient because the terms can be multiplied together to see the combined effect.

$$\begin{aligned}(1-B)(1-B^m)y_t &= (1-B-B^m+B^{m+1})y_t \\ &= y_t - y_{t-1} - y_{t-m} + y_{t-m-1}.\end{aligned}$$

For monthly data, $m = 12$ and we obtain the same result as earlier.

Outline

1 Stationarity and differencing

2 Transformations and adjustments

3 Moving averages

4 Classical Decomposition

5 A tidy forecasting workflow

6 Benchmark methods

Transformations and adjustments

Purpose: simplify the patterns in the historical data by removing known sources of variation or by making the pattern more consistent across the whole data set.

Simpler patterns usually lead to more accurate forecasts!

Types:

1. Mathematical transformations,
2. Calendar adjustments,
3. Population adjustments, and
4. Inflation adjustments.

Variance Stabilisation

If the data show different variation at different levels of the series, then a transformation can be useful.

Denote original observations as $y_1, \ldots, y_T$ and transformed observations as $w_1, \ldots, w_T$.

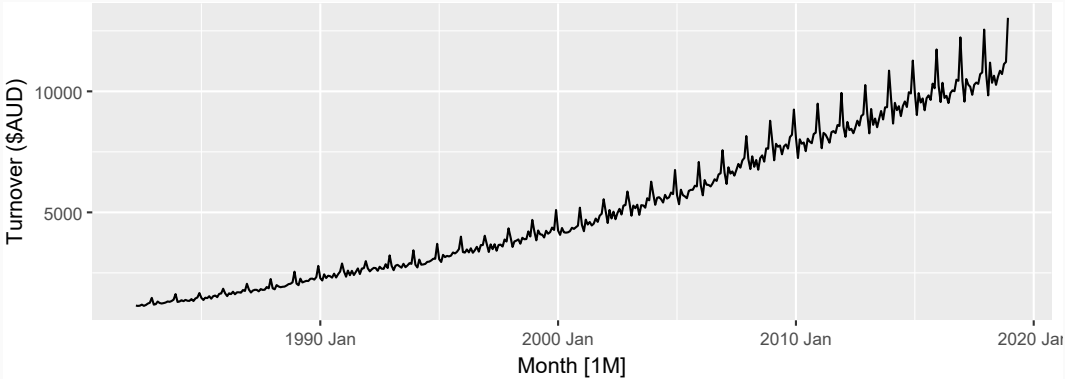
Mathematical transformations for stabilising variation

Square root	$w_t = \sqrt{y_t}$	$\downarrow$
Cube root	$w_t = \sqrt[3]{y_t}$	Increasing
Logarithm	$w_t = \log(y_t)$	strength

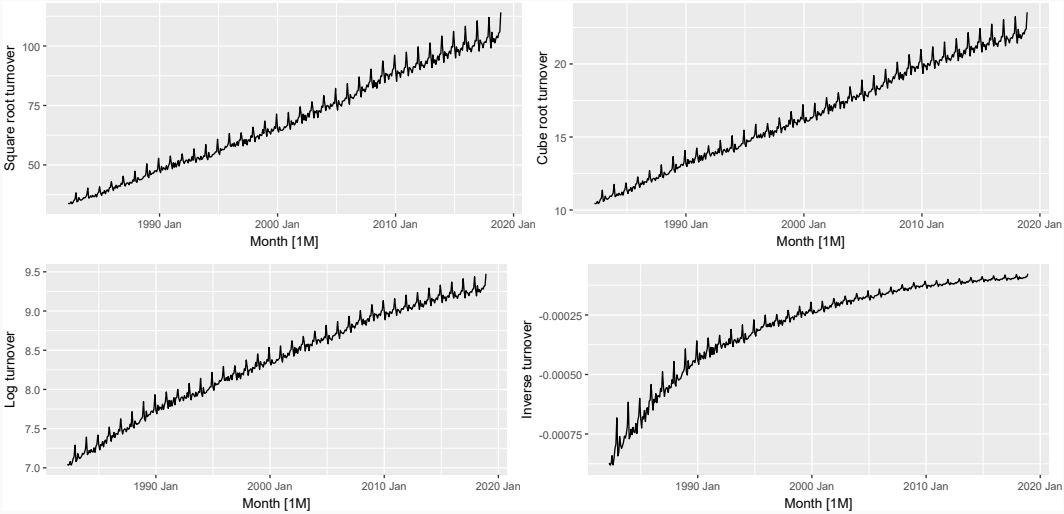
Logarithms, in particular, are useful because they are more interpretable: changes in a log value are **relative (percent) changes on the original scale**.

Variance Stabilisation

```
food <- aus_retail |>
  filter(Industry == "Food retailing") |>
  summarise(Turnover = sum(Turnover))
```



Variance Stabilisation



Box-Cox transformations

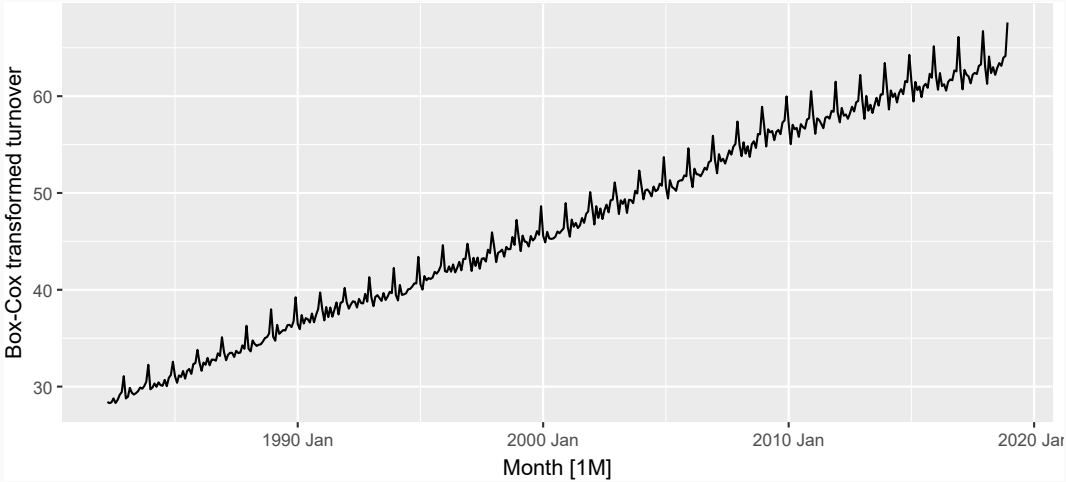
Each of these transformations is close to a member of the family of **Box-Cox transformations**:

$$w_t = \begin{cases} \log(y_t), & \lambda = 0; \\ (y_t^\lambda - 1)/\lambda, & \lambda \neq 0. \end{cases}$$

- $\lambda = 1$: (No substantive transformation)
- $\lambda = \frac{1}{2}$: (Square root plus linear transformation)
- $\lambda = 0$: (Natural logarithm)
- $\lambda = -1$: (Inverse plus 1)

Box-Cox transformations

```
food |> autoplot(box_cox(Turnover, 1/3)) +
  labs(y = "Box-Cox transformed turnover")
```



Box-Cox transformations

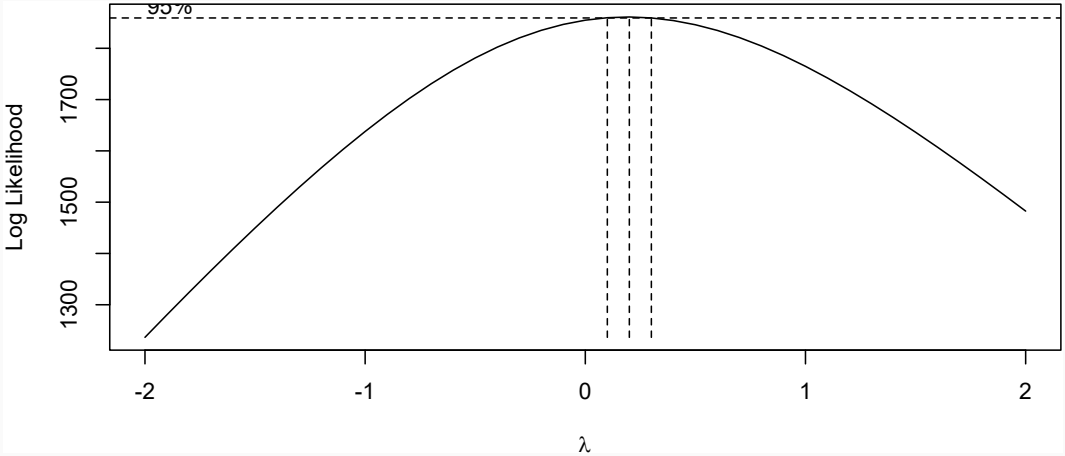
```
food |>
  features(Turnover, features = guerrero)
```

```
# A tibble: 1 x 1
  lambda_guerrero
            <dbl>
1          0.0895
```

```
# The `guerrero` approach attempts to balance the seasonal fluctuations and random v
# Always check the results.
# A low value of $\lambda$ can give extremely large prediction intervals.
```

Box-Cox transformations

```
library(TSA)
lambda.fit <- BoxCox.ar(food$Turnover)
```



Back-transformation

We must reverse the transformation (or *back-transform*) to obtain forecasts on the original scale. The reverse Box-Cox transformations are given by

$$y_t = \begin{cases} \exp(w_t), & \lambda = 0; \\ (\lambda W_t + 1)^{1/\lambda}, & \lambda \neq 0. \end{cases}$$

Box-Cox transformations

- y_t^λ for λ close to zero behaves like logs.
- If some $y_t = 0$, then must have $\lambda > 0$
- if some $y_t < 0$, no power transformation is possible unless all y_t adjusted by **adding a constant to all values**.
- Simple values of λ are easier to explain.
- Results are relatively insensitive to λ.
- Often no transformation ($\lambda = 1$) needed.
- Transformation can have very large effect on PI.
- Choosing $\lambda = 0$ is a simple way to force forecasts to be positive

Outline

1 Stationarity and differencing
2 Transformations and adjustments
3 Moving averages
4 Classical Decomposition
5 A tidy forecasting workflow
6 Benchmark methods

Moving averages

The simplest estimate of the trend-cycle uses **moving averages**.

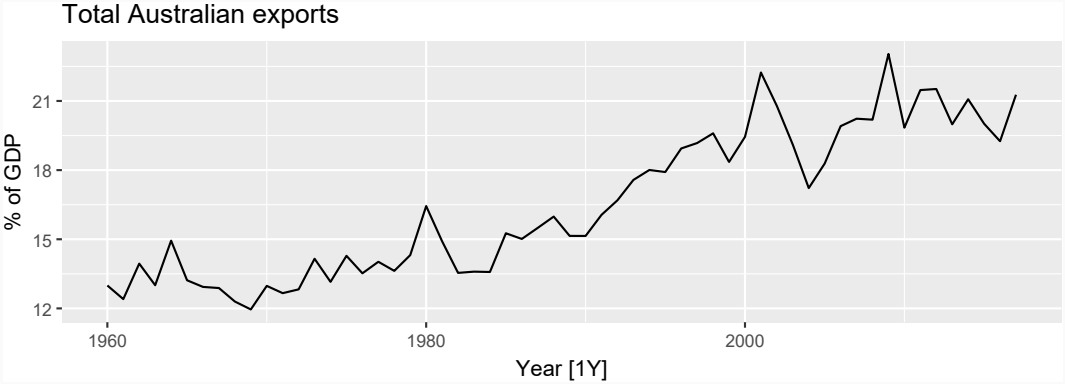
m-MA

$$\hat{T}_t = \frac{1}{m} \sum_{j=-k}^{k} y_{t+j}$$

where $k = \frac{m-1}{2}$.

Moving averages

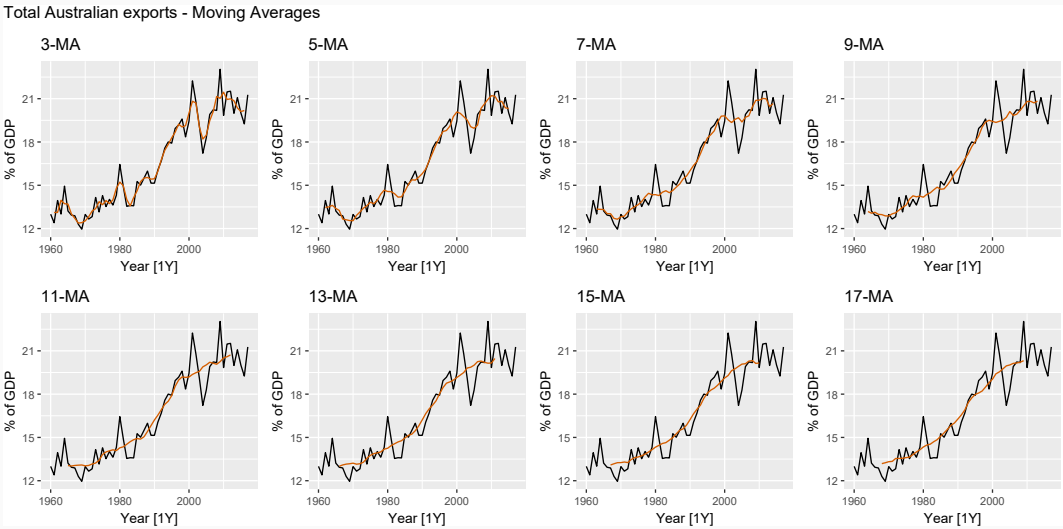
```
global_economy |> filter(Country == "Australia") |>
  autoplot(Exports) +
  labs(y="% of GDP", title= "Total Australian exports")
```



Moving average smoothing

Year	Exports	5-MA
1960.00	12.99	
1961.00	12.40	
1962.00	13.94	13.46
1963.00	13.01	13.50
1964.00	14.94	13.61
...	...	...
2012.00	21.52	20.78
2013.00	19.99	20.81
2014.00	21.08	20.37
2015.00	20.01	20.32
2016.00	19.25	
2017.00	21.27	

Moving average smoothing



Moving average smoothing

So a moving average is an average of nearby points

- observations nearby in time are also likely to be close in value.
- average eliminates some randomness in the data, leaving a smooth trend-cycle component.

$$\text{3-MA:} \quad \hat{T}_t = (y_{t-1} + y_t + y_{t+1})/3$$

$$\text{5-MA:} \quad \hat{T}_t = (y_{t-2} + y_{t-1} + y_t + y_{t+1} + y_{t+2})/5$$

- each average computed by dropping oldest observation and including next observation.
- averaging moves through time series until trend-cycle computed at each observation possible.

Endpoints

Why is there no estimate at ends?

- For a 3 MA, there cannot be estimates at time 1 or time T because the observations at time 0 and $T + 1$ are not available.
- Generally: there cannot be estimates at times near the endpoints.

The order of the MA

- larger order means smoother, flatter curve
- larger order means more points lost at ends
- order = length of season *or cycle* removes pattern
- But so far odd orders?

Centered MA

4 MA:

$$\frac{1}{4}(y_{t-2} + y_{t-1} + y_t + y_{t+1})$$

or

$$\frac{1}{4}(y_{t-1} + y_t + y_{t+1} + y_{t+2})$$

Solution: take a further 2-MA to "centre" result.

$$\begin{aligned} T_t &= \frac{1}{2}\left\{\frac{1}{4}(y_{t-2} + y_{t-1} + y_t + y_{t+1}) \right. \\ &\qquad \left. + \frac{1}{4}(y_{t-1} + y_t + y_{t+1} + y_{t+2})\right\} \\ &= \frac{1}{8}y_{t-2} + \frac{1}{4}y_{t-1} + \frac{1}{4}y_t + \frac{1}{4}y_{t+1} + \frac{1}{8}y_{t+2} \end{aligned}$$

Centered MA

Year	Data	4-MA	2×4-MA
1992 Q1	443.00		
1992 Q2	410.00	451.25	
1992 Q3	420.00		450.00
1992 Q4	532.00	448.75	450.12
1993 Q1	433.00	451.50	450.25
1993 Q2	421.00	449.00	446.50
1993 Q3	410.00	444.00	446.00
1993 Q4	512.00	448.00	443.00
⋮	⋮	438.00 ⋮	⋮

Moving average trend-cycle

A moving average of the same length as the season removes the seasonal pattern.

- For quarterly data: use a 2×4 MA
- For monthly data: use a 2×12 MA

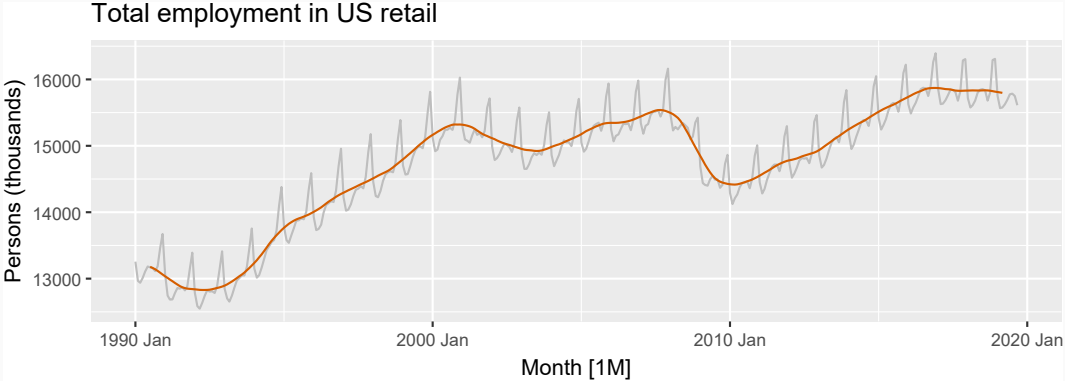
$$\hat{T}_t = \frac{1}{24}y_{t-6} + \frac{1}{12}y_{t-5} + \cdots + \frac{1}{12}y_{t+5} + \frac{1}{24}y_{t+6}$$

Moving average trend-cycle

```
us_retail_employment_ma <- us_retail_employment |>
  mutate(
    `12-MA` = slider::slide_dbl(Employed, mean,
          .before = 5, .after = 6, .complete = TRUE),
    `2x12-MA` = slider::slide_dbl(`12-MA`, mean,
          .before = 1, .after = 0, .complete = TRUE)
  )

us_retail_employment_ma |>
  autoplot(Employed, color = "gray") +
  autolayer(us_retail_employment_ma, vars(`2x12-MA`),
            color = "#D55E00") +
  labs(y = "Persons (thousands)",
       title = "Total employment in US retail")
```

Moving average trend-cycle



Outline

1 Stationarity and differencing

2 Transformations and adjustments

3 Moving averages

4 Classical Decomposition

5 A tidy forecasting workflow

6 Benchmark methods

Trend-cycle

Additive decomposition: $y_t = T_t + S_t + R_t = \hat{T}_t + \hat{S}_t + \hat{R}_t$
Multiplicative decomposition: $y_t = T_t \times S_t \times R_t = \hat{T}_t \times \hat{S}_t \times \hat{R}_t$

- Estimate $\hat{T}$ using $(2 \times m)$-MA if m is even. Otherwise, estimate $\hat{T}$ using m-MA

Compute de-trended series

- Additive decomposition: $y_t - \hat{T}_t$
- Multiplicative decomposition: $y_t / \hat{T}_t$

De-trending

Remove smoothed series $\hat{T}_t$ from y_t to leave S_t and R_t.

- Additive model: $y_t - \hat{T}_t = (\hat{T}_t + \hat{S}_t + \hat{R}_t) - \hat{T}_t = \hat{S}_t + \hat{R}_t$
- Multiplicative model: $\frac{y_t}{\hat{T}_t} = \frac{\hat{T}_t \times \hat{S}_t \times \hat{R}_t}{\hat{T}_t} = \hat{S}_t \times \hat{R}_t$

Estimating seasonal component

- Seasonal index for each season is estimated as an average of the detrended series for that season of successive years.
- E.g., take averages across all Januaries to get $S^{(1)}$ if your data is monthly.
- If necessary, adjust the seasonal indices so that:
  - for additive: $S^{(1)} + S^{(2)} + \ldots + S^{(12)} = 0$
  - for multiplicative: $S^{(1)} + S^{(2)} + \ldots + S^{(12)} = m$
- The seasonal component $\hat{S}_t$ simply consists of replications of the seasonal indices.

Remainder component

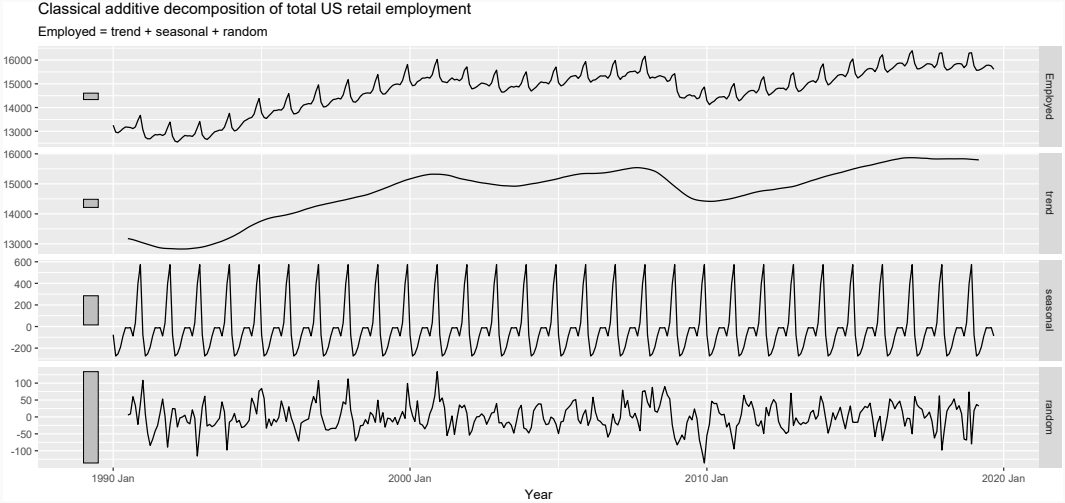
Additive decomposition: $\hat{R}_t = y_t - \hat{T}_t - \hat{S}_t$

Multiplicative decomposition: $\hat{R}_t = y_t/(\hat{T}_t\hat{S}_t)$

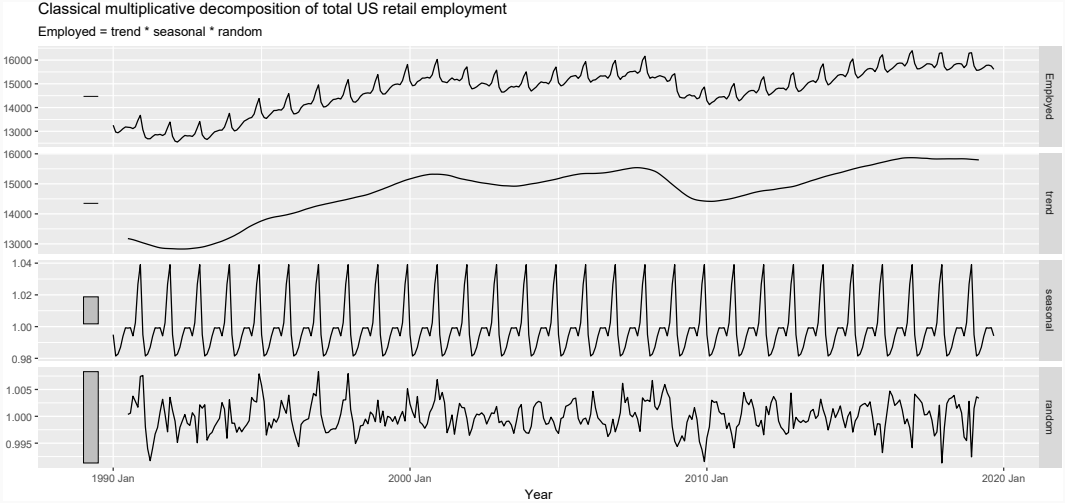
Classical decomposition

- Choose additive or multiplicative depending on which gives the most stable components.
- For multiplicative model, this method of estimation is known as ratio-to-moving-average method.

US Retail Employment



US Retail Employment



US Retail Employment

```
us_retail_employment |>
  model(classical_decomposition(Employed, type = "additive")) |>
  components() |>
  autoplot() + xlab("Year") +
  ggtitle("Classical additive decomposition of total
          US retail employment")
```

Comments on classical decomposition

- Estimate of trend is unavailable for first few and last few observations.
- Seasonal component repeats from year to year. May not be realistic.
- Not robust to outliers.
- Newer methods designed to overcome these problems.

Outline

1 Stationarity and differencing

2 Transformations and adjustments

3 Moving averages

4 Classical Decomposition

5 A tidy forecasting workflow

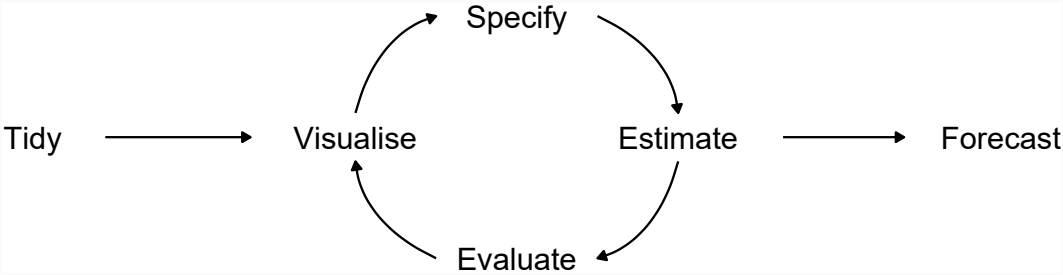
6 Benchmark methods

A tidy forecasting workflow

The process of producing forecasts can be split up into a few fundamental steps.

1. Preparing data
2. Data visualisation
3. Specifying a model
4. Model estimation
5. Accuracy & performance evaluation
6. Producing forecasts

A tidy forecasting workflow



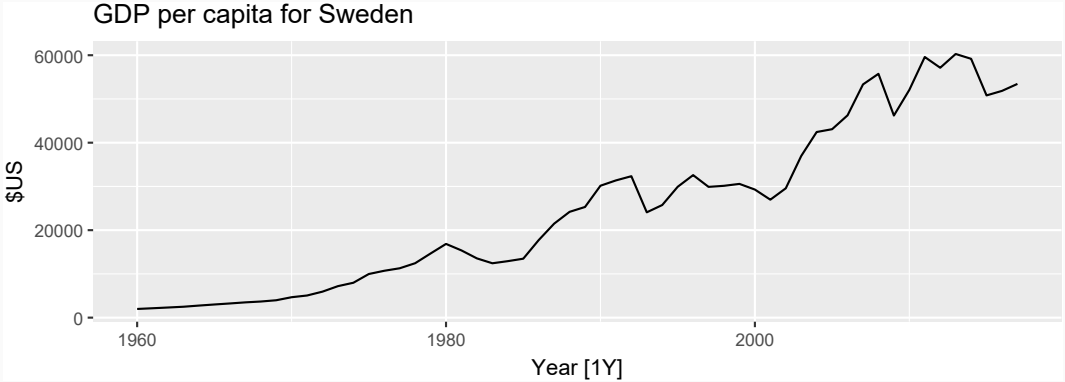
Data preparation (tidy)

```
gdppc <- global_economy |>
  mutate(GDP_per_capita = GDP / Population) |>
  select(Year, Country, GDP, Population, GDP_per_capita)
gdppc
```

```
# A tsibble: 15,150 x 5 [1Y]
# Key:       Country [263]
    Year Country              GDP Population GDP_per_capita
   <dbl> <fct>              <dbl>      <dbl>          <dbl>
 1  1960 Afghanistan  537777811.    8996351           59.8
 2  1961 Afghanistan  548888896.    9166764           59.9
 3  1962 Afghanistan  546666678.    9345868           58.5
 4  1963 Afghanistan  751111191.    9533954           78.8
 5  1964 Afghanistan  800000044.    9731361           82.2
 6  1965 Afghanistan 1006666638.    9938414          101.
 7  1966 Afghanistan 1399999967.   10152331          138.
```

Data visualisation

```
gdppc |>
  filter(Country == "Sweden") |>
  autoplot(GDP_per_capita) +
  labs(title = "GDP per capita for Sweden", y = "$US")
```



Model estimation

The `model()` function trains models to data.

```
fit <- gdppc |>
  model(trend_model = TSLM(GDP_per_capita ~ trend()))
fit
```

```
# A mable: 263 x 2
# Key:     Country [263]
   Country             trend_model
   <fct>                   <model>
 1 Afghanistan              <TSLM>
 2 Albania                  <TSLM>
 3 Algeria                  <TSLM>
 4 American Samoa           <TSLM>
 5 Andorra                  <TSLM>
 6 Angola                   <TSLM>
```

Model estimation

The `model()` function trains models to data.

```
fit <- gdppc |>
  model(trend_model = TSLM(GDP_per_capita ~ trend()))
fit
```

```
# A mable: 263 x 2
# Key:     Country [263]
   Country              trend_model
   <fct>                     <model>
 1 Afghanistan               <TSLM>
 2 Albania                   <TSLM>
 3 Algeria                   <TSLM>
 4 American Samoa            <TSLM>
 5 Andorra                   <TSLM>
 6 Angola                    <TSLM>
```

A `mable` is a model table, each cell corresponds to a fitted model.

Producing forecasts

```
fit |> forecast(h = "3 years")
```

```
# A fable: 789 x 5 [1Y]
# Key:     Country, .model [263]
   Country        .model       Year
   <fct>          <chr>       <dbl>
 1 Afghanistan    trend_model  2018
 2 Afghanistan    trend_model  2019
 3 Afghanistan    trend_model  2020
 4 Albania        trend_model  2018
 5 Albania        trend_model  2019
 6 Albania        trend_model  2020
 7 Algeria        trend_model  2018
 8 Algeria        trend_model  2019
 9 Algeria        trend_model  2020
```

Producing forecasts

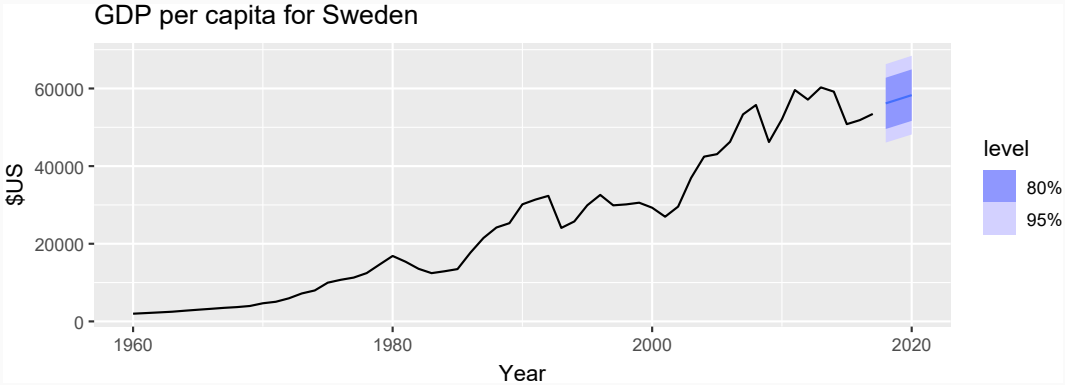
```
fit |> forecast(h = "3 years")
```

```
# A fable: 789 x 5 [1Y]
# Key:     Country, .model [263]
   Country        .model       Year
   <fct>          <chr>       <dbl>
 1 Afghanistan    trend_model  2018
 2 Afghanistan    trend_model  2019
 3 Afghanistan    trend_model  2020
 4 Albania        trend_model  2018
 5 Albania        trend_model  2019
 6 Albania        trend_model  2020
 7 Algeria        trend_model  2018
 8 Algeria        trend_model  2019
 9 Algeria        trend_model  2020
```

A `fable` is a forecast table with point forecasts and distributions.

Visualising forecasts

```
fit |>
  forecast(h = "3 years") |>
  filter(Country == "Sweden") |>
  autoplot(gdppc) + labs(title = "GDP per capita for Sweden", y = "$US")
```



Recap

The process of producing forecasts can be split up into a few fundamental steps.

1. Preparing data - generate a `tsibble`
2. Data visualisation
3. Specifying a model
4. Model estimation - `model() ->  mable`
5. Accuracy & performance evaluation
6. Producing forecasts - `forecast() -> fable`

Outline

1 Stationarity and differencing
2 Transformations and adjustments
3 Moving averages
4 Classical Decomposition
5 A tidy forecasting workflow
6 Benchmark methods

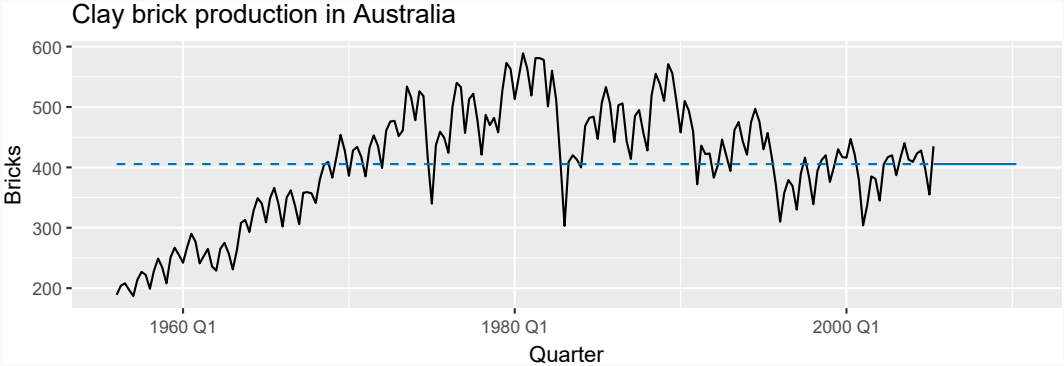
Some simple forecasting methods

Common simple and effective methods:

1. Average method
2. Naïve method
3. Seasonal naïve method
4. Drift method

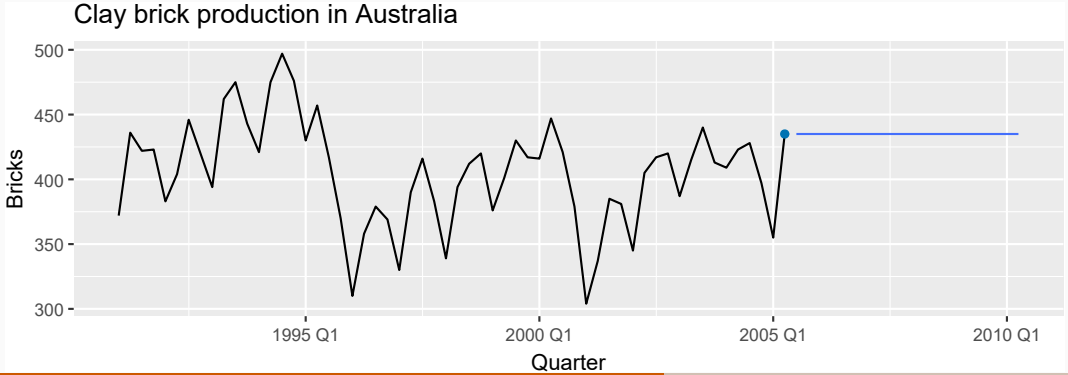
`MEAN(y)`: Average method

- Forecast of all future values is equal to mean of historical data $\{y_1, \ldots, y_T\}$.
- Forecasts: $\hat{y}_{T+h|T} = \bar{y} = (y_1 + \cdots + y_T)/T$



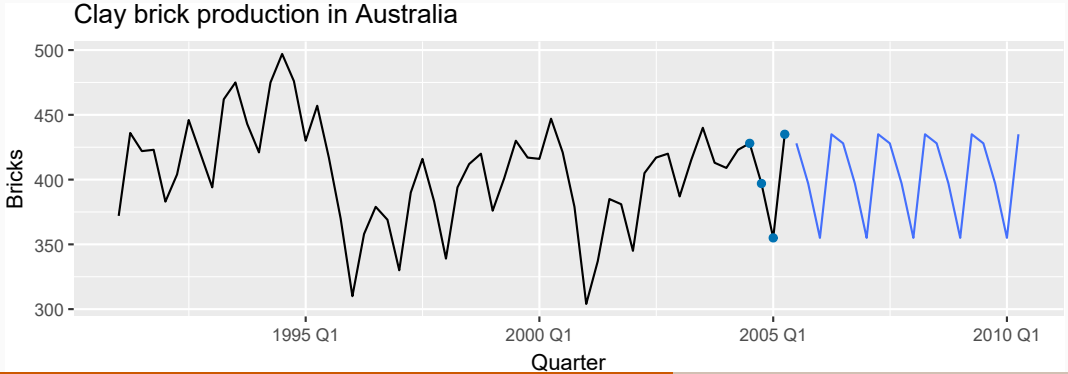
`NAIVE(y)`: Naïve method

- Forecasts equal to last observed value.
- Forecasts: $\hat{y}_{T+h|T} = y_T$.
- Consequence of efficient market hypothesis.



`SNAIVE(y ~ lag(m))`: Seasonal naïve method

- Forecasts equal to last value from same season.
- Forecasts: $\hat{y}_{T+h|T} = y_{T+h-m(k+1)}$, where m = seasonal period and k is the integer part of $(h-1)/m$.



`RW(y ~ drift())`: Drift method

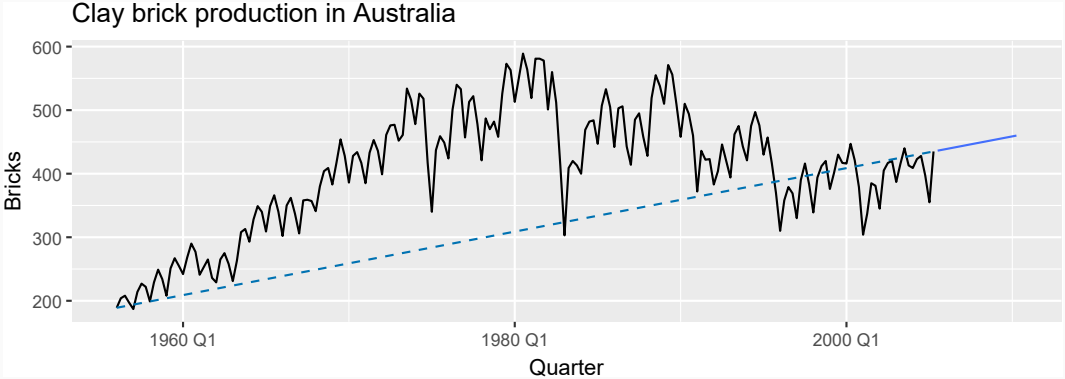
- Forecasts equal to last value plus average change.
- Forecasts:

$$\begin{aligned}\hat{y}_{T+h|T} &= y_T + \frac{h}{T-1}\sum_{t=2}^{T}(y_t - y_{t-1}) \\ &= y_T + \frac{h}{T-1}(y_T - y_1).\end{aligned}$$

- Equivalent to extrapolating a line drawn between first and last observations.

Some simple forecasting methods

Drift method



Recall: Random walk model

If differenced series is white noise with zero mean:

$$y_t - y_{t-1} = \varepsilon_t \quad \text{or} \quad y_t = y_{t-1} + \varepsilon_t$$

where $\varepsilon_t \sim NID(0, \sigma^2)$.

- Very widely used for non-stationary data.
- This is the model behind the naïve method.
- Random walks typically have:
  - long periods of apparent trends up or down
  - Sudden/unpredictable changes in direction
- Forecast are equal to the last observation (naïve)
  - future movements up or down are equally likely.

Note: Random walk with drift model

If differenced series is white noise with non-zero mean:

$$y_t - y_{t-1} = c + \varepsilon_t \quad \text{or} \quad y_t = c + y_{t-1} + \varepsilon_t$$

where $\varepsilon_t \sim NID(0, \sigma^2)$.

- c is the average change between consecutive observations.
- If $c > 0$, y_t will tend to drift upwards and vice versa.
- This is the model behind the drift method.

Recall: Seasonal differencing

If seasonally differenced data is white noise it implies:

$$y_t - y_{t-m} = \varepsilon_t \quad \text{or} \quad y_t = y_{t-m} + \varepsilon_t$$

where $\varepsilon_t \sim NID(0, \sigma^2)$.

- The model behind the seasonal naïve method.

Other statistical and econometric models

- Exponential smoothing models
- Time-series regression models
- Autoregressive integrated moving average (ARIMA) models
- ARCH/GARCH models
- State-space models
- Many others!!!

Simple forecasting methods - Model fitting

The `model()` function trains models to data.

```
brick_fit <- aus_production |>
  filter(!is.na(Bricks)) |>
  model(
    Seasonal_naive = SNAIVE(Bricks),
    Naive = NAIVE(Bricks),
    Drift = RW(Bricks ~ drift()),
    Mean = MEAN(Bricks)
  )
```

```
# A mable: 1 x 4
  Seasonal_naive   Naive          Drift    Mean
          <model> <model>        <model> <model>
1       <SNAIVE> <NAIVE> <RW w/ drift>  <MEAN>
```

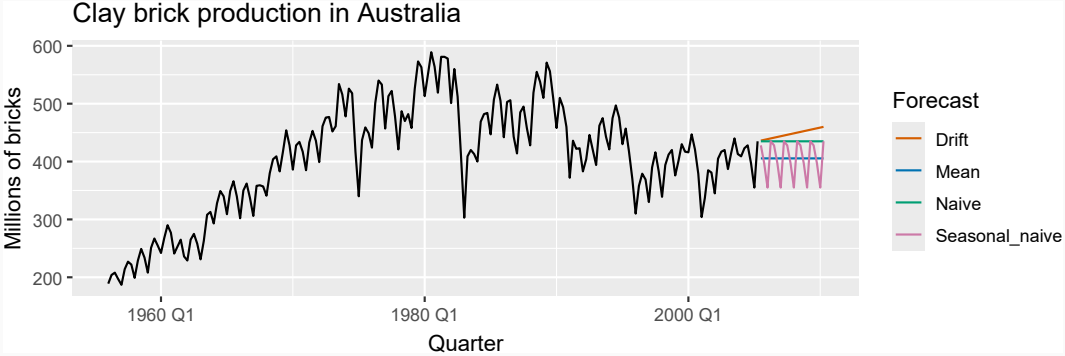
Simple forecasting methods - Producing forecasts

```
brick_fc <- brick_fit |>
  forecast(h = "5 years")
```

```
# A fable: 80 x 4 [1Q]
# Key:     .model [4]
  .model          Quarter
  <chr>             <qtr>
1 Seasonal_naive 2005 Q3
2 Seasonal_naive 2005 Q4
3 Seasonal_naive 2006 Q1
4 Seasonal_naive 2006 Q2
# i 76 more rows
# i 2 more variables: Bricks <dist>, .mean <dbl>
```

Simple forecasting methods - Visualising forecasts

```
brick_fc |>
  autoplot(aus_production, level = NULL) +
  labs(title = "Clay brick production in Australia",
       y = "Millions of bricks") +
  guides(colour = guide_legend(title = "Forecast"))
```



Facebook closing stock price

```
# Extract training data
fb_stock <- gafa_stock |>
  filter(Symbol == "FB") |>
  mutate(trading_day = row_number()) |>
  update_tsibble(index = trading_day, regular = TRUE)

# Specify, estimate and forecast
fb_stock |>
  model(
    Mean = MEAN(Close),
    Naive = NAIVE(Close),
    Drift = RW(Close ~ drift())
  ) |>
  forecast(h = 42) |>
  autoplot(fb_stock, level = NULL) +
  labs(title = "Facebook closing stock price", y = "$US") +
  guides(colour = guide_legend(title = "Forecast"))
```

Facebook closing stock price



Fitted values

- $\hat{y}_{t|t-1}$ is the forecast of y_t based on observations $y_1, \ldots, y_{t-1}$.
- We call these “fitted values”.
- Sometimes drop the subscript: $\hat{y}_t \equiv \hat{y}_{t|t-1}$.
- Often not true forecasts since parameters are estimated on all data.

For example:

- $\hat{y}_t = \bar{y}$ for average method.
- $\hat{y}_t = y_{t-1} + (y_T - y_1)/(T-1)$ for drift method.

Forecasting residuals

Residuals in forecasting: difference between observed value and its fitted value: $e_t = y_t - \hat{y}_{t|t-1}$.

Assumptions

1. $\{e_t\}$ uncorrelated. If they aren't, then information left in residuals that should be used in computing forecasts.
2. $\{e_t\}$ have mean zero. If they don't, then forecasts are biased.

Useful properties (for distributions & prediction intervals)

3. $\{e_t\}$ have constant variance.
4. $\{e_t\}$ are normally distributed.

Facebook closing stock price

Facebook closing stock price

```
fit <- fb_stock |> model(NAIVE(Close))
augment(fit)
```

```
# A tsibble: 1,258 x 7 [1]
# Key:       Symbol, .model [1]
   Symbol .model       trading_day Close .fitted .resid .innov
   <chr>  <chr>              <int> <dbl>   <dbl>  <dbl>  <dbl>
 1 FB     NAIVE(Close)           1  54.7    NA    NA      NA
 2 FB     NAIVE(Close)           2  54.6    54.7 -0.150 -0.150
 3 FB     NAIVE(Close)           3  57.2    54.6  2.64   2.64
 4 FB     NAIVE(Close)           4  57.9    57.2  0.720  0.720
 5 FB     NAIVE(Close)           5  58.2    57.9  0.310  0.310
 6 FB     NAIVE(Close)           6  57.2    58.2 -1.01  -1.01
 7 FB     NAIVE(Close)           7  57.9    57.2  0.720  0.720
 8 FB     NAIVE(Close)           8  55.9    57.9 -2.03  -2.03
 9 FB     NAIVE(Close)           9  57.7    55.9  1.83   1.83
10 FB     NAIVE(Close)          10  57.6    57.7 -0.140 -0.140
# i 1,248 more rows
```

Facebook closing stock price

```
fit <- fb_stock |> model(NAIVE(Close))
augment(fit)
```

```
# A tsibble: 1,258 x 7 [1]
# Key:       Symbol, .model [1]
   Symbol .model       trading_day Close .fitted   .resid  .innov
   <chr>  <chr>              <int> <dbl>   <dbl>    <dbl>   <dbl>
 1 FB     NAIVE(Close)           1  54.7      NA       NA      NA
 2 FB     NAIVE(Close)           2  54.6    54.7   -0.150  -0.150
 3 FB     NAIVE(Close)           3  57.2    54.6    2.64    2.64
 4 FB     NAIVE(Close)           4  57.9    57.2    0.720   0.720
 5 FB     NAIVE(Close)           5  58.2    57.9    0.310   0.310
 6 FB     NAIVE(Close)           6  57.2    58.2   -1.01   -1.01
                                 7  57.9    57.2    0.720   0.720
                                 8  55.9    57.9   -2.03   -2.03
                                 9  57.7    55.9    1.83    1.83
                                10  57.6    57.7   -0.140  -0.140
```

$\hat{y}_{t|t-1}$ e_t

Naïve forecasts:

$$\hat{y}_{t|t-1} = y_{t-1}$$

$$e_t = y_t - \hat{y}_{t|t-1} = y_t - y_{t-1}$$

Facebook closing stock price

```
augment(fit) |>
  ggplot(aes(x = trading_day)) +
  geom_line(aes(y = Close, colour = "Data")) +
  geom_line(aes(y = .fitted, colour = "Fitted"))
```



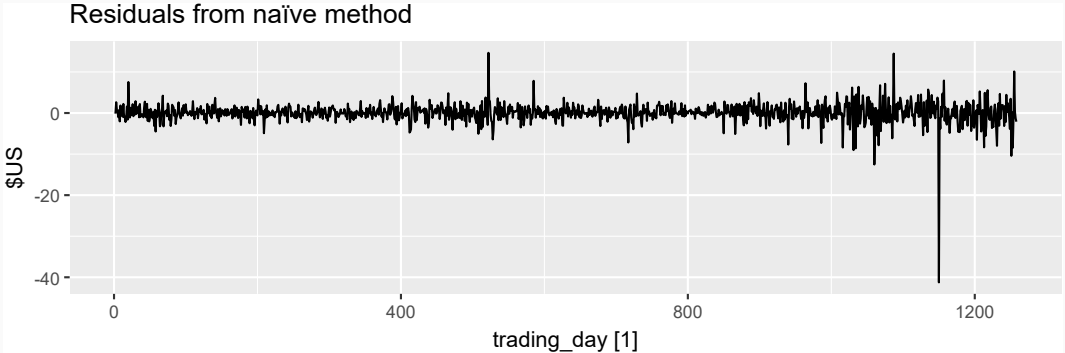
Facebook closing stock price

```
augment(fit) |>
  filter(trading_day > 1100) |>
  ggplot(aes(x = trading_day)) +
  geom_line(aes(y = Close, colour = "Data")) +
  geom_line(aes(y = .fitted, colour = "Fitted"))
```



Facebook closing stock price

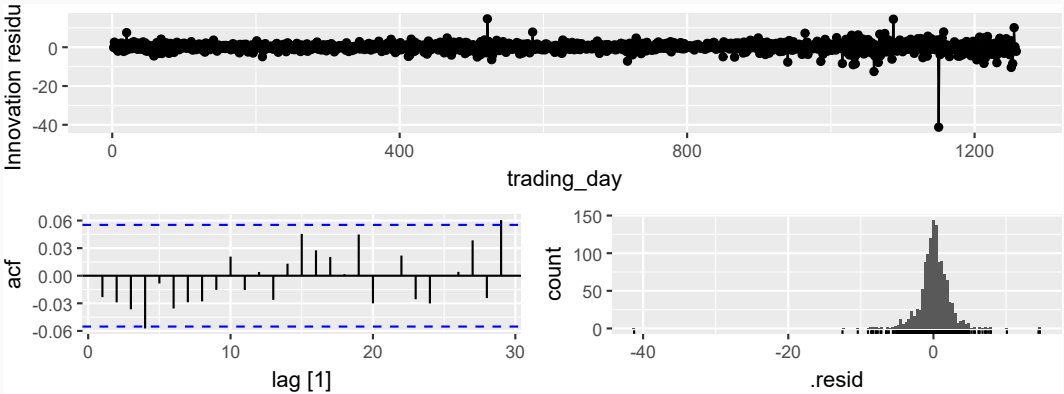
```
augment(fit) |>
  autoplot(.resid) +
  labs(y = "$US",
       title = "Residuals from naïve method")
```



Can also obtain Histogram of residuals and ACF plot

gg_tsresiduals() function

```
gg_tsresiduals(fit)
```



ACF of residuals

- We assume that the residuals are white noise (uncorrelated, mean zero, constant variance). If they aren't, then there is information left in the residuals that should be used in computing forecasts.
- So a standard residual diagnostic is to check the ACF of the residuals of a forecasting method.
- We *expect* these to look like white noise.

Portmanteau tests

r_k = autocorrelation of residual at lag k

Consider a *whole set* of r_k values, and develop a test to see whether the set is significantly different from a zero set.

Box-Pierce test

$$Q = T\sum_{k=1}^{\ell} r_k^2$$

where ℓ is max lag being considered and T is number of observations.

- If each r_k close to zero, Q will be **small**.
- If some r_k values large (positive or negative), Q will be **large**.

Portmanteau tests

r_k = autocorrelation of residual at lag k

Consider a *whole set* of r_k values, and develop a test to see whether the set is significantly different from a zero set.

Ljung-Box test

$$Q^* = T(T+2)\sum_{k=1}^{\ell}(T-k)^{-1}r_k^2$$

where ℓ is max lag being considered and T is number of observations.

- My preferences: $\ell = 10$ for non-seasonal data, $h = 2m$ for seasonal data (where m is seasonal period).
- Better performance, especially in small samples.

Portmanteau tests

- If data are WN, Q^* has χ^2 distribution with ℓ degrees of freedom.
- `lag` = ℓ

```
augment(fit) |>
  features(.resid, ljung_box, lag = 10)
```

```
# A tibble: 1 x 4
  Symbol .model       lb_stat lb_pvalue
  <chr>  <chr>          <dbl>     <dbl>
1 FB     NAIVE(Close)    12.1     0.276
```

Forecast distributions

- A forecast $\hat{y}_{T+h|T}$ is (usually) the mean of the conditional distribution $y_{T+h} \mid y_1, \dots, y_T$.
- Most time series models produce normally distributed forecasts.
- The forecast distribution describes the probability of observing any future value.

Forecast distributions

Assuming residuals are normal, uncorrelated, sd = $\hat{\sigma}$:

Mean:	$y_{T+h\|T} \sim N(\bar{y}, (1+1/T)\hat{\sigma}^2)$
Naïve:	$y_{T+h\|T} \sim N(y_T, h\hat{\sigma}^2)$
Seasonal naïve:	$y_{T+h\|T} \sim N(y_{T+h-m(k+1)}, (k+1)\hat{\sigma}^2)$
Drift:	$y_{T+h\|T} \sim N(y_T + \frac{h}{T-1}(y_T - y_1), h\frac{T+h}{T}\hat{\sigma}^2)$

where k is the integer part of $(h-1)/m$.

Note that when $h = 1$ and T is large, these all give the same approximate forecast variance: $\hat{\sigma}^2$.

Prediction intervals

- A prediction interval gives a region within which we expect y_{T+h} to lie with a specified probability.
- Assuming forecast errors are normally distributed, then a 95% PI is

$$\hat{y}_{T+h|T} \pm 1.96\hat{\sigma}_h$$

where $\hat{\sigma}_h$ is the st dev of the h-step distribution.

- When $h = 1$, $\hat{\sigma}_h$ can be estimated from the residuals.

Prediction intervals

```
aus_production |>
  filter(!is.na(Bricks)) |>
  model(Seasonal_naive = SNAIVE(Bricks)) |>
  forecast(h = "5 years") |>
  hilo(level = 95)  |>
  mutate(lower = `95%`$lower, upper=`95%`$upper)
```

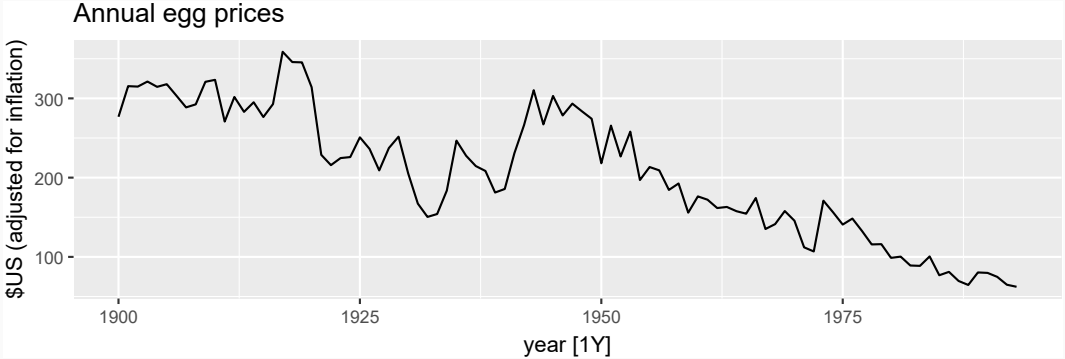
```
# A tsibble: 20 x 7 [1Q]
# Key:       .model [1]
   .model          Quarter
   <chr>             <qtr>
 1 Seasonal_naive 2005 Q3
 2 Seasonal_naive 2005 Q4
 3 Seasonal_naive 2006 Q1
 4 Seasonal_naive 2006 Q2
 5 Seasonal_naive 2006 Q3
 6 Seasonal_naive 2006 Q4
```

Prediction intervals

- Point forecasts are often useless without a measure of uncertainty (such as prediction intervals).
- Prediction intervals require a stochastic model (with random errors, etc).
- For most models, prediction intervals get wider as the forecast horizon increases.
- Use `level` argument to control coverage.
- Check residual assumptions before believing them.
- Prediction intervals are usually too narrow due to unaccounted uncertainty.

Note: Modelling with transformations

```
eggs <- prices |>
  filter(!is.na(eggs)) |>
  select(eggs)
eggs |> autoplot() +
  labs(title = "Annual egg prices", y = "$US (adjusted for inflation)")
```



Note: Modelling with transformations

Transformations used in the left of the formula will be automatically back-transformed. To model log-transformed egg prices, you could use:

```
fit <- eggs |>
  model(RW(log(eggs) ~ drift()))
fit
```

```
# A mable: 1 x 1
  `RW(log(eggs) ~ drift())`
                     <model>
1              <RW w/ drift>
```

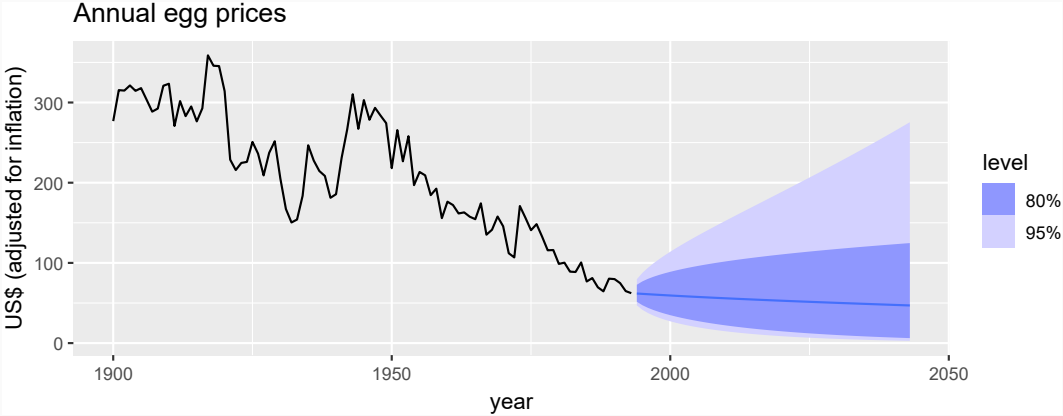
Note: Forecasting with transformations

```
fc <- fit |>
  forecast(h = 50)
fc
```

```
# A fable: 50 x 4 [1Y]
# Key:     .model [1]
   .model                      year             eggs .mean
   <chr>                      <dbl>           <dist> <dbl>
 1 RW(log(eggs) ~ drift())  1994 t(N(4.1, 0.018))  61.8
 2 RW(log(eggs) ~ drift())  1995 t(N(4.1, 0.036))  61.4
 3 RW(log(eggs) ~ drift())  1996 t(N(4.1, 0.055))  61.0
 4 RW(log(eggs) ~ drift())  1997 t(N(4.1, 0.074))  60.6
 5 RW(log(eggs) ~ drift())  1998 t(N(4.1, 0.093))  60.2
 6 RW(log(eggs) ~ drift())  1999    t(N(4, 0.11))  59.8
 7 RW(log(eggs) ~ drift())  2000    t(N(4, 0.13))  59.4
 8 RW(log(eggs) ~ drift())  2001    t(N(4, 0.15))  59.0
 9 RW(log(eggs) ~ drift())  2002    t(N(4, 0.18))  58.6
```

Note: Forecasting with transformations

```
fc |> autoplot(eggs) +
  labs(title = "Annual egg prices",
       y = "US$ (adjusted for inflation)")
```



Note: Bias adjustment

- Back-transformed point forecasts are medians.
- Back-transformed PI have the correct coverage.

Back-transformed means

Let X have mean μ and variance σ^2.

Let $f(x)$ be back-transformation function, and $Y = f(X)$.

Taylor series expansion about μ:

$$Y = f(X) \approx f(\mu) + (X-\mu)f'(\mu) + \frac{1}{2}(X-\mu)^2 f''(\mu).$$

$$\mathrm{E}[Y] = \mathrm{E}[f(X)] \approx f(\mu) + \tfrac{1}{2}\sigma^2 f''(\mu)$$

Training and test sets



- A model which fits the training data well will not necessarily forecast well.
- A perfect fit can always be obtained by using a model with enough parameters.
- Over-fitting a model to data is just as bad as failing to identify a systematic pattern in the data.
- The test set must not be used for *any* aspect of model development or calculation of forecasts.
- Forecast accuracy is based only on the test set.

Recall: Forecast errors

Forecast “error”: the difference between an observed value and its forecast.

$$e_{T+h} = y_{T+h} - \hat{y}_{T+h|T},$$

where the training data is given by $\{y_1, \dots, y_T\}$

- Unlike residuals, forecast errors on the test set involve multi-step forecasts.
- These are *true* forecast errors as the test data is not used in computing $\hat{y}_{T+h|T}$.

Measures of forecast accuracy

y_{T+h} = $(T+h)$th observation, $h = 1, \ldots, H$

$\hat{y}_{T+h|T}$ = its forecast based on data up to time T.

e_{T+h} = $y_{T+h} - \hat{y}_{T+h|T}$

$$\text{MAE} = \text{mean}(|e_{T+h}|)$$

$$\text{MSE} = \text{mean}(e_{T+h}^2) \qquad \text{RMSE} = \sqrt{\text{mean}(e_{T+h}^2)}$$

$$\text{MAPE} = 100\text{mean}(|e_{T+h}|/|y_{T+h}|)$$

- MAE, MSE, RMSE are all scale dependent.
  - A good way to choose the best forecasting model is to find the model with the smallest RMSE computed using time series cross-validation.
- MAPE is scale independent but is only sensible if $y_t \gg 0$ for all t, and y has a natural zero.

Mean Absolute Scaled Error

$$\text{MASE} = \text{mean}(|e_{T+h}|/Q)$$

- For non-seasonal series, scale uses naïve forecasts:

$$Q = \frac{1}{T-1}\sum_{t=2}^{T}|y_t - y_{t-1}|$$

- For seasonal series, scale uses seasonal naïve forecasts:

$$Q = \frac{1}{T-m}\sum_{t=m+1}^{T}|y_t - y_{t-m}|$$

where m is the seasonal frequency

Proposed by Hyndman and Koehler (IJF, 2006).

Root Mean Squared Scaled Error

$$\text{RMSSE} = \sqrt{\text{mean}(e_{T+h}^2/Q)}$$

- For non-seasonal series, scale uses naïve forecasts:

$$Q = \frac{1}{T-1}\sum_{t=2}^{T}(y_t - y_{t-1})^2$$

- For seasonal series, scale uses seasonal naïve forecasts:

$$Q = \frac{1}{T-m}\sum_{t=m+1}^{T}(y_t - y_{t-m})^2$$

where m is the seasonal frequencyq

Proposed by Hyndman and Koehler (IJF, 2006).

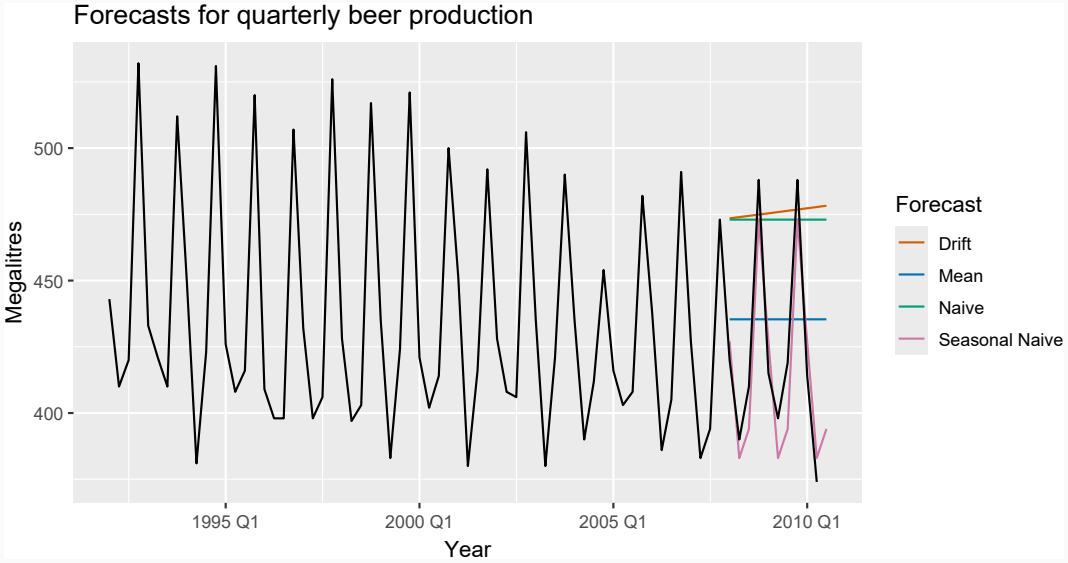
Measures of forecast accuracy

```
train <- aus_production |>
  filter(between(year(Quarter), 1992, 2007))

beer <- aus_production |>
  filter(year(Quarter) >= 1992)

beer_fc_plot <- train |>
  model(
    Mean = MEAN(Beer),
    `Naive` = NAIVE(Beer),
    `Seasonal Naive` = SNAIVE(Beer),
    Drift = RW(Beer ~ drift())
  ) |>
  forecast(h=11) |>
  autoplot(beer, level = NULL) +
    ggtitle("Forecasts for quarterly beer production") +
    xlab("Year") + ylab("Megalitres") +
    guides(colour=guide_legend(title="Forecast"))
```

Measures of forecast accuracy



Training set accuracy

```
recent_production <- aus_production |>
  filter(year(Quarter) >= 1992)
train <- recent_production |> filter(year(Quarter) <= 2007)
beer_fit <- train |>
  model(
    Mean = MEAN(Beer),
    `Naive` = NAIVE(Beer),
    `Seasonal Naive` = SNAIVE(Beer),
    Drift = RW(Beer ~ drift())
  )
accuracy(beer_fit)
```

	RMSE	MAE	MAPE	MASE
Mean method	43.6	35.2	7.89	2.46
Naive method	65.3	54.7	12.16	3.83
Seasonal Naive method	16.8	14.3	3.31	1.00
Drift method	65.3	54.8	12.18	3.83

Test set accuracy

```
beer_fc <- beer_fit |>
  forecast(h = 10)
accuracy(beer_fc, recent_production)
```

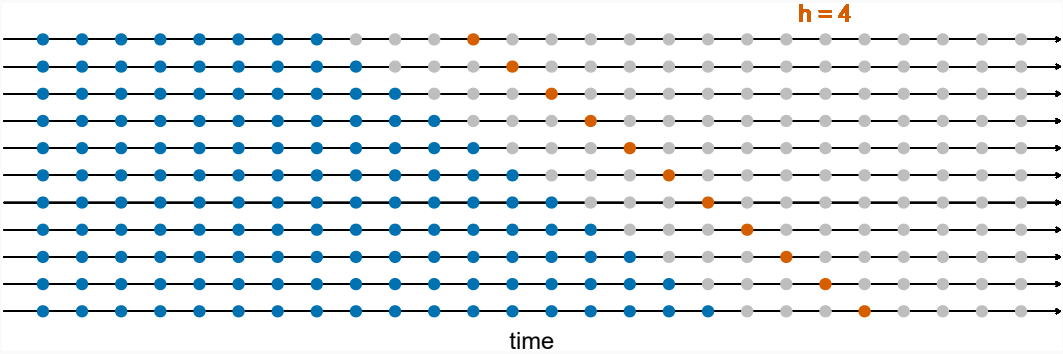
	RMSE	MAE	MAPE	MASE
Drift method	64.9	58.9	14.58	4.117
Mean method	38.4	34.8	8.28	2.435
Naive method	62.7	57.4	14.18	4.014
Seasonal Naive method	14.3	13.4	3.17	0.937

Time series cross-validation

Traditional evaluation



Time series cross-validation

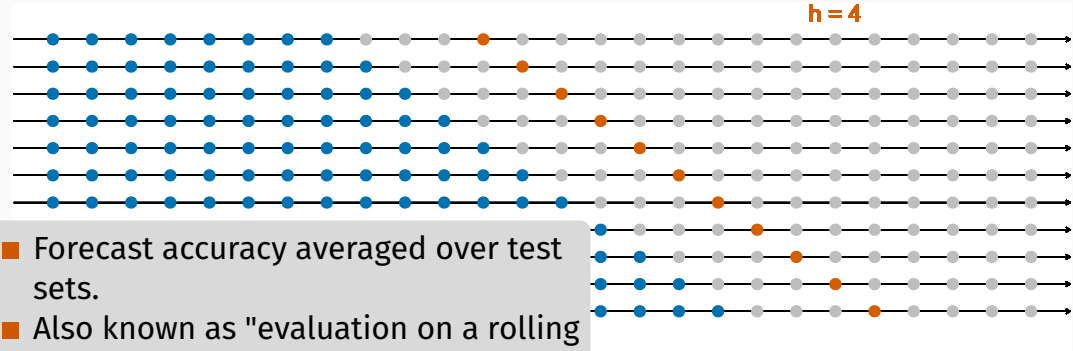


Time series cross-validation

Traditional evaluation



Time series cross-validation



- Forecast accuracy averaged over test sets.
- Also known as "evaluation on a rolling forecasting origin"

Creating the rolling training sets

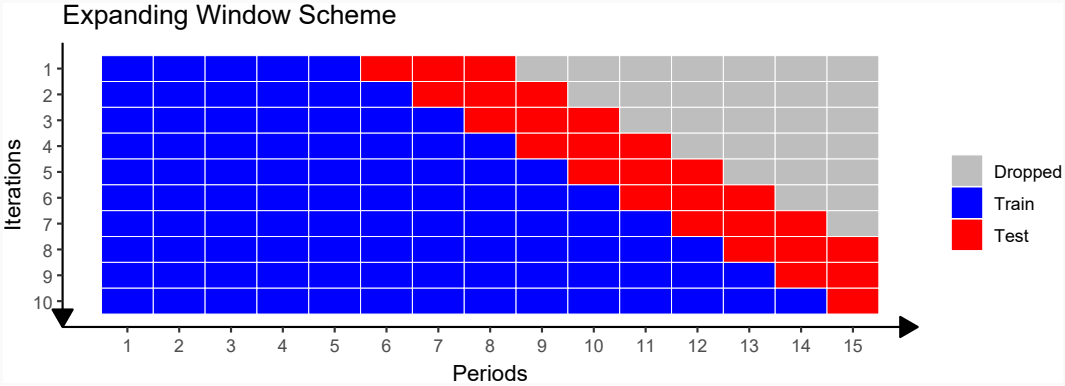
There are three main rolling types which can be used.

- Stretch: extends a growing length window with new data.
- Slide: shifts a fixed length window through the data.
- Tile: moves a fixed length window without overlap.

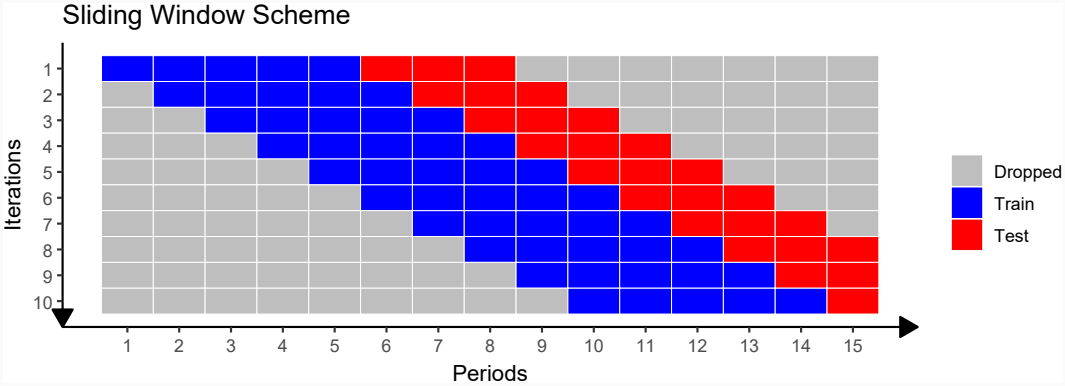
Three functions to roll a tsibble: `stretch_tsibble()`, `slide_tsibble()`, and `tile_tsibble()`.

For time series cross-validation, stretching windows are most commonly used.

Expanding Window Scheme (stretch)



Sliding Window Scheme



Time series cross-validation

Stretch with a minimum length of 3, growing by 1 each step.

```
fb_stretch <- fb_stock |>
  stretch_tsibble(.init = 3, .step = 1) |>
  filter(.id != max(.id))
```

```
# A tsibble: 790,650 x 4 [1]
# Key:       .id [1,255]
  Date       Close trading_day   .id
  <date>     <dbl>       <int> <int>
1 2014-01-02  54.7           1     1
2 2014-01-03  54.6           2     1
3 2014-01-06  57.2           3     1
4 2014-01-02  54.7           1     2
5 2014-01-03  54.6           2     2
6 2014-01-06  57.2           3     2
7 2014-01-07  57.9           4     2
# i 790,643 more rows
```

Time series cross-validation

Estimate RW w/ drift models for each window.

```
fit_cv <- fb_stretch |>
  model(RW(Close ~ drift()))
```

```
# A mable: 1,255 x 3
# Key:     .id, Symbol [1,255]
    .id Symbol `RW(Close ~ drift())`
  <int> <chr>                <model>
1     1 FB             <RW w/ drift>
2     2 FB             <RW w/ drift>
3     3 FB             <RW w/ drift>
4     4 FB             <RW w/ drift>
# i 1,251 more rows
```

Time series cross-validation

Produce one step ahead forecasts from all models.

```
fc_cv <- fit_cv |>
  forecast(h = 1)
```

```
# A fable: 1,255 x 5 [1]
# Key:     .id, Symbol [1,255]
    .id Symbol trading_day
  <int> <chr>        <dbl>
1     1 FB               4
2     2 FB               5
3     3 FB               6
4     4 FB               7
# i 1,251 more rows
# i 2 more variables: Close <dist>, .mean <dbl>
```

Time series cross-validation

```
# Cross-validated
fc_cv |> accuracy(fb_stock)
# Training set
fb_stock |>
  model(RW(Close ~ drift())) |>
  accuracy()
```

	RMSE	MAE	MAPE
Cross-validation	2.418	1.469	1.266
Training	2.414	1.465	1.261

Key reference

Hyndman, R. J. & Athanasopoulos, G. (2021) *Forecasting: principles and practice*, 3rd edition

OTexts.org/fpp3/

- Free and online
- Data sets in associated R packages
- R code for examples